Image Forgery Detection on the CG-1050 Dataset

Model Architecture, Training Methodology & Layer Breakdown — Xception (Transfer Learning)

1. Project Overview & Objective

This document describes the design of the **Xception** model within a comparative-analysis project — "*Comparative Analysis of Transfer-Learning-Based Deep Learning Models for Image Forgery Detection*" — evaluated on the CG-1050 dataset. Xception, EfficientNet-B0 and ResNet50 are trained under the identical preprocessing, split and evaluation protocol described here, so that their results (Accuracy, Precision, Recall, F1-score, ROC-AUC, training time and inference time) can be compared fairly. This document covers the Xception model specifically: its architecture, why it fits this task, how it is trained, and its internal layer structure.

The task is framed as classification, not localization: given an input image, predict whether it is *authentic* or *tampered* and, for tampered images, which of four manipulation types was used. Pixel-level localization (marking exactly where in the image the tampering occurred) was considered and intentionally excluded — see Section 4.

2. Dataset Overview — CG-1050

CG-1050 (Castro, Ballesteros & Renza, 2019) is a purpose-built image-forgery dataset consisting of a small set of original photographs and a much larger set of manipulated derivatives, each labelled with its tampering type via an accompanying description file.

Property	Value
Original (authentic) images	100 — 15 colour, 85 grayscale
Tampered images	1050 total, derived from the 100 originals (~10–11 manipulated versions per source)
Tampering types (Stage 2 classes)	Copy-move: 338 · Colorizing: 354 · Retouching: 308 · Cut-paste: 50
Class balance (Stage 1)	Authentic : Tampered ≈ 1 : 10.5 — significant imbalance
Ground-truth masks	Provided per tampered image, but not used in this classification-only pipeline (see Section 4)
Labelling source	Accompanying description spreadsheet mapping each tampered file to its source image ID and tampering type

The 100:1050 source-to-derivative ratio is the single most important property of this dataset for experiment design: it is what makes a random image-level train/test split unsafe (Section 4) and what makes plain accuracy misleading (Section 4).

3. Model Architecture

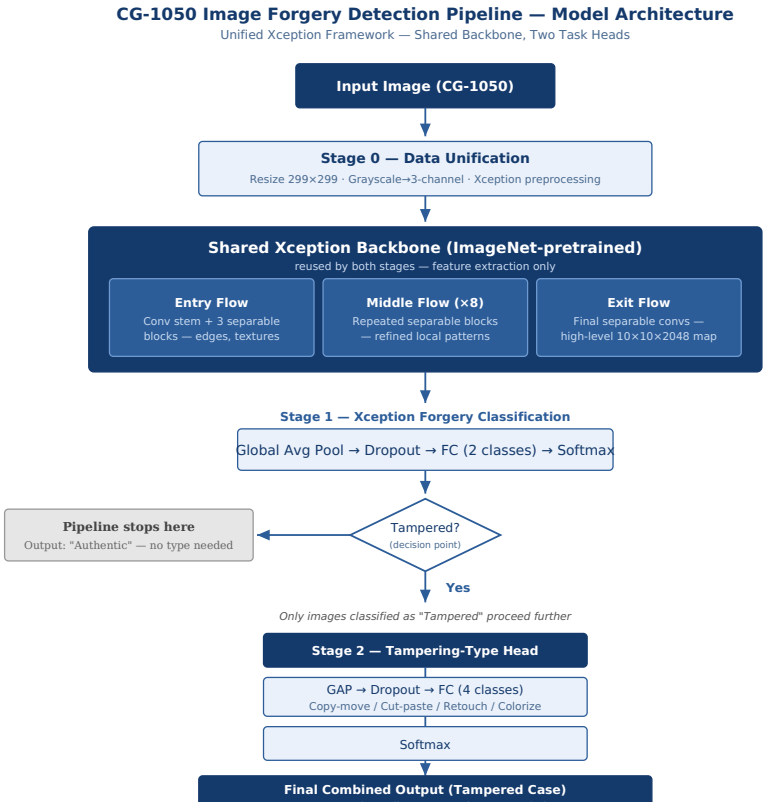
The pipeline uses a single model family — **Xception** (depthwise separable convolutional network, ImageNet-pretrained) — across both stages, in two task-specific variants. This keeps the entire pipeline under one backbone, one preprocessing function, and one training/inference API, rather than mixing a different architecture per stage.

- **Stage 1 – Forgery Classification:** Xception (pretrained), fine-tuned with a 2-class head (*authentic* / *tampered*) on the combined CG-1050 image pool.
- **Stage 2 – Tampering-Type Classification:** Xception (pretrained), retrained with a 4-class head (*copy-move* / *cut-paste* / *retouching* / *colorizing*), predicting the manipulation type directly from the image rather than relying on the description spreadsheet at inference time.

Trade-off: *ResNet50 has a longer track record as a general-purpose backbone and EfficientNet-B0 is considerably cheaper to train; this design favours Xception's depthwise separable convolutions, which separate spatial from channel-wise filtering and are better suited to the fine-grained texture and boundary artifacts left by tampering, at the cost of higher GPU memory use and slower fine-tuning than EfficientNet-B0.*

Pixel-level forgery localization (predicting a tampering mask) was considered as a third stage but is left out of this design: it requires a full encoder–decoder segmentation network, a Dice/IoU evaluation pipeline, and pixel-aligned mask supervision — a substantially larger scope than the classification comparison this project targets. It is noted here as a natural extension, not part of the current pipeline.

4. Architecture Diagram



Dataset: CG-1050 (Castro, Ballesteros & Renza, 2019) — 100 original · 1050 tampered · Image Forgery Detection — Transfer Learning with Xception

Note: Stage 1 runs first and acts as a gate. Only images classified as "tampered" proceed to Stage 2, which predicts the manipulation type. Localization (pixel-level masking) is intentionally excluded from this pipeline — see Section 1.

5. Step-by-Step Explanation

Step 0 — Data Unification

Before anything reaches the model, every CG-1050 image is resized to 299×299 (Xception's native input size) and converted to a consistent 3-channel RGB format. This matters here because the original directory holds 15 colour and 85 grayscale images: each single-channel grayscale image is replicated across three channels so the pretrained ImageNet filters receive the input shape they expect. Pixel values are then scaled to the [-1, 1] range using Xception's own `preprocess_input` function rather than a plain 0–1 rescale. This ensures the model sees consistent input rather than images that vary in size, channel count, or intensity range.

Step 1 — Shared Backbone (Feature Extraction)

The image passes through the Xception backbone, which does not make any decision yet — it extracts visual features from the image:

- **Entry Flow** — a standard convolutional stem followed by three depthwise-separable blocks; picks up basic visual information: edges, gradients, colour transitions and rough textures.
- **Middle Flow** — the same separable block repeated eight times at constant resolution; builds on those basics to learn more complex patterns, such as inconsistent texture regions, unnatural object boundaries and duplicated structures.
- **Exit Flow** — the final separable convolutions that expand the channel depth to 2048; captures high-level, abstract evidence of manipulation across the whole image.

At the end of this step, the image has been converted into a rich internal feature representation (a 10×10×2048 feature map) — not a final answer yet.

Step 2 — Stage 1: Forgery Classification

Using the extracted features, a classification head decides whether the image is tampered or authentic:

- **Global Average Pooling** compresses the 10×10×2048 feature map into a 2048-value summary
- **Dropout** randomly deactivates part of that summary during training, which limits over-fitting on a dataset that only contains 100 distinct source scenes
- **A Fully Connected layer** converts the summary into 2 raw scores
- **Softmax** turns those scores into probabilities (e.g. 94% tampered, 6% authentic)

This is the decision gate. If the result is "authentic," the pipeline stops here — there is no manipulation to classify or localize, so the remaining two stages are not run.

Step 3 — Stage 2 (only if tampered)

If Stage 1 predicts "tampered," the image is passed to a second head that answers "what kind of manipulation is this?" using the same pooling → dropout → dense → softmax structure as Stage 1, but with 4 output classes (copy-move / cut-paste / retouching / colorizing) instead of 2. This stage is skipped entirely for images Stage 1 already classified as authentic, since there is no manipulation type to assign.

Step 4 — Final Combined Output

The result from Stage 2 is combined with the Stage 1 verdict: the final output is the "Tampered" / "Authentic" label, and — only for tampered images — the predicted tampering type with its confidence score.

6. Training Methodology

Data preparation (Stage 0)

All images are resized to 299×299, converted to 3-channel RGB, and normalised with Xception's preprocessing. The description spreadsheet supplied with CG-1050 is parsed to attach two labels to every tampered file — its source original image and its tampering type — which drives both the Stage 2 labels and the splitting strategy below.

Splits — grouped by source image (critical)

CG-1050 contains 1050 tampered images derived from only 100 source scenes, i.e. roughly 10–11 manipulations per original. A random image-level split would therefore place manipulated versions of the same scene in both the training and test sets, and the reported accuracy would reflect scene memorisation rather than forgery detection.

> **Splitting rule:** splits are drawn at the level of the *source image ID*, not the individual file. Roughly 70 / 15 / 15 of the 100 originals are assigned to train / validation / test, and every tampered child and mask follows its parent into the same subset. Splits are additionally stratified so that the colour/grayscale ratio and the four tampering types remain represented in each subset.

Class imbalance

The authentic : tampered ratio is 100 : 1050 (about 1 : 10.5), so a model that predicts "tampered" for everything would already score about 91% accuracy. This is handled by class weighting in the loss (or by oversampling the authentic class), and accuracy is never reported on its own — balanced accuracy, per-class recall and the confusion matrix are reported alongside it. The four tampering types are also uneven (338 copy-move, 354 colorizing, 308 retouching, 50 cut-paste), so

Stage 2 uses class weighting as well, and per-class recall on the small cut-paste class is watched specifically.

Stage 1 training

Xception fine-tuned in two phases. **Phase 1:** the backbone is frozen and only the new classification head is trained (Adam, learning rate 1e-3) until the head stabilises. **Phase 2:** the exit flow and the last few middle-flow blocks are unfrozen and trained with a much lower learning rate (1e-5) so the pretrained ImageNet filters are refined rather than destroyed; batch-normalisation layers are kept in inference mode during fine-tuning. Binary cross-entropy loss, early stopping on validation loss, and a learning-rate-on-plateau schedule are used throughout.

Hyperparameters

Setting	Value
Input size	299 × 299 × 3
Base weights	ImageNet (pretrained), no top layer
Optimizer	Adam
Learning rate — head training (Phase 1)	1e-3
Learning rate — fine-tuning (Phase 2)	1e-5
Batch size	16–32 (source-grouped, so batches never mix train/val/test scenes)
Loss — Stage 1 / Stage 2	Binary cross-entropy (class-weighted) / Categorical cross-entropy (class-weighted)
Regularisation	Dropout (0.3–0.5) before the final dense layer; early stopping on validation loss
Epochs	Head training ~10–15; fine-tuning ~20–30, governed by early stopping

Augmentation (chosen conservatively)

Horizontal flips, small rotations and mild scale jitter are applied. Heavy blur, strong denoising, aggressive JPEG re-compression and strong colour jitter are deliberately avoided: forgery detection depends on exactly the fine noise, compression and boundary traces that those operations erase, so an over-aggressive augmentation pipeline would remove the evidence the model is supposed to learn.

Stage 2 training

Xception fine-tuned on the 1050 tampered images only, with a 4-class head, using the same grouped splits, the same two-phase schedule and categorical cross-entropy loss. The encoder is initialised from the Stage 1 fine-tuned weights rather than from ImageNet directly, since it has already adapted to the visual domain of CG-1050.

Evaluation

Stage 1 — accuracy, balanced accuracy, precision, recall, F1 and ROC-AUC, with attention to recall on the authentic (minority) class; **Stage 2** — per-class precision/recall/F1 and a 4×4 confusion matrix. Training time and per-image inference time are also recorded, so Xception can later be compared against EfficientNet-B0 and ResNet50 under an identical protocol.

Inference flow

An input image is preprocessed and classified (Stage 1); if tampered, it is passed to Stage 2 for tampering-type prediction; the final output combines both labels.

> **Known dataset caveat to report honestly:** in CG-1050 the colorizing manipulation is applied to grayscale originals, which means a model could separate "colorized" from the other classes using the colour statistics of the image alone rather than any forensic trace. The colour/grayscale composition of each split is therefore recorded and reported, and Stage 2 results are additionally reported on the grayscale-only subset as a sanity check.

7. Layer Breakdown

Both stages share the same Xception backbone for extracting visual features from the image, and each attaches its own classification head on top to output a category.

5.1 Shared Backbone (used by both stages)

Layer Type	Function
Conv layers (Entry Flow stem)	Two standard convolutional layers (32 and 64 filters, stride 2) that shrink the image and extract simple features — edges, gradients, basic textures
Depthwise Separable Conv blocks	Xception's core building block. A standard convolution is split into a *depthwise* step (one filter per input channel, learning spatial pattern) and a *pointwise* 1×1 step (mixing channels). This decoupling gives fine-grained spatial sensitivity with far fewer parameters — well matched to the subtle texture and boundary artifacts that tampering leaves behind
Residual (skip) connections	A parallel 1×1 convolution added around each block, so gradients flow directly through the 36-layer stack and deep training stays stable — the same idea ResNet introduced, reused inside Xception
Middle Flow (×8 repetition)	Eight identical separable blocks at constant resolution (19×19×728); the depth where most of the discriminative pattern learning happens, without further loss of spatial detail
Exit Flow	Final separable convolutions that expand the representation to 2048 channels at 10×10 resolution — the high-level summary passed to every head
Batch Normalisation + ReLU	Applied after each convolution; stabilises and speeds up training. Kept frozen in inference mode during fine-tuning so the pretrained running statistics are not corrupted by small batches

5.2 Stage 1 & Stage 2 — Xception Classification Heads

After the shared backbone, each stage adds an identical head structure, differing only in the number of output classes:

Layer Type	Function
Global Average Pooling	Compresses the final 10×10×2048 feature map into a single 2048-value summary — flattens the image understanding into a decision-ready format, with no extra parameters
Dropout	Randomly zeroes a fraction of that summary during training; important on CG-1050 because only 100 distinct scenes exist and the model would otherwise memorise them
Fully Connected (Dense) layer	Final decision layer — outputs 2 values for Stage 1 (authentic / tampered) or 4 values for Stage 2 (copy-move / cut-paste / retouching / colorizing)
Softmax	Converts the raw output values into probabilities that sum to 100%, giving a clean confidence score for the top prediction

Note: exact layer counts are fixed for Xception (36 convolutional layers organised into 14 modules, all but the first and last with residual connections). The table above describes the structural pattern, not an exact layer-by-layer count.

8. Limitations & Future Scope

- **Small number of source scenes (100).** All tampered images derive from only 100 originals, so the model's exposure to real-world scene diversity is limited; results should be read as evidence of forensic-pattern learning on this dataset, not as a claim of general real-world robustness.
- **Class imbalance.** The 1:10.5 authentic-to-tampered ratio and the small cut-paste class (50 images) mean accuracy alone is not a reliable metric; balanced accuracy, per-class recall and the confusion matrix are reported for this reason (Section 6).
- **Colorizing shortcut risk.** Because colorizing is applied only to grayscale originals, a model could learn to separate that class from colour statistics rather than genuine manipulation traces (Section 6, evaluation callout).
- **No pixel-level localization.** This pipeline classifies but does not localize tampered regions. Adding a segmentation stage (using the ground-truth masks CG-1050 already provides) is a natural next step if the project later needs to show *where* an image was altered, not just *whether*.
- **Single-dataset evaluation.** All results are specific to CG-1050; testing on a second forgery dataset would be needed to assess generalisation before any real-world use.

Dataset: CG-1050 — Castro, M., Ballesteros, D.M. & Renza, D. (2019), *A dataset of 1050-tampered color and grayscale images*, Data in Brief 28. 100 original images (15 colour, 85 grayscale), 1050 tampered images (338 copy-move, 50 cut-paste, 308 retouching, 354 colorizing). Pipeline scope: classification only (Stages 1–2); pixel-level mask localization not included.